OpenClaw Custom Agent File Structure Guide

Audience: Developers creating new custom agents for the OpenClaw ecosystem.
Purpose: This document defines the standard folder/file structure every custom agent must follow. When you generate a new agent, produce files matching this layout. OpenClaw will discover, register, and run the agent based on these conventions.

Quick Reference - Directory Tree

```
&lt;agent-root&gt;/
|-- openclaw.json                    # Agent manifest (REQUIRED)
|-- .env.example                     # Environment variable template
|-- .gitignore                       # Standard ignores
|
|-- workspace/
|   |-- SOUL.md                      # Agent persona + workflow orchestration (REQUIRED)
|   +-- IDENTITY.md                  # Agent branding &amp; identity card (REQUIRED)
|
|-- skills/
|   |-- data-ingestion-openclaw/
|   |   +-- SKILL.md                 # Always-included: Data Ingestion API reference
|   |-- &lt;custom-skill-1&gt;/
|   |   +-- SKILL.md                 # One SKILL.md per capability
|   +-- ...
|
|-- cron/
|   +-- &lt;job-name&gt;.json              # OpenClaw cron job definitions
|
|-- workflows/
|   +-- main.yaml                    # Lobster workflow DAG definition
|
|-- check-environment.sh             # Dev helper: validate env setup
|-- install-dependencies.sh          # Dev helper: install system deps
+-- test-workflow.sh                 # Dev helper: manual e2e test
```

---

1. openclaw.json - Agent Manifest

Role: The entry point OpenClaw reads to register the agent. It tells the platform what this agent is, which skills it has, and where to find them.

How OpenClaw uses it: On startup (or `openclaw gateway restart`), the platform scans for `openclaw.json`, loads the agent ID, resolves skill directories, and makes the agent available for cron triggers, manual invocations, and workflow execution.

Schema

```
{
  "agentId": "&lt;kebab-case-unique-id&gt;",
  "version": "1.0.0",
  "description": "&lt;one-line summary of what the agent does&gt;",
  "agents": [
    {
```

```
      "id": "main",
      "skills": [
        "data-ingestion-openclaw",
        "&lt;custom-skill-folder-name-1&gt;",
        "&lt;custom-skill-folder-name-2&gt;"
      ]
    }
  ],
  "skills": {
    "dirs": ["skills", "workspace/skills"]
  },
  "workflows": {
    "dirs": ["workflows"]
  }
}
```

Rules

Field	Required	Notes
agentId	Yes	Must be globally unique across all OpenClaw agents. Use kebab-case.
version	Yes	Semver. Bump on every release.
agents[].id	Yes	Typically "main". Multi-agent setups can have multiple entries.
agents[].skills	Yes	Array of skill folder names. Order does not matter - orchestration order is defined in SOUL.md.
skills.dirs	Yes	Directories OpenClaw scans for <folder>/SKILL.md files. Always include "skills".
workflows.dirs	Yes	Directories OpenClaw scans for workflow YAML files.

---

2. workspace/ - Agent Brain

The `workspace/` folder holds the agent's personality, behavior rules, and identity. OpenClaw loads these files into the agent's context at runtime.

2a. workspace/SOUL.md - Persona & Workflow Orchestration

Role: This is the most important file in the agent. It defines:

* Who the agent is (persona, tone, style)
* What it does (purpose, key behaviors)
* How it orchestrates work (step-by-step workflow when triggered)
* Error handling rules
* Data flow diagram
* Privacy constraints

How OpenClaw uses it: When the agent receives a trigger (cron or manual), OpenClaw loads SOUL.md into the LLM context. The agent then follows the instructions in SOUL.md to execute its workflow - reading SKILL.md files, calling `exec()` for bash/python, and calling `message()` for delivery.

What to include in SOUL.md

```
# &lt;Agent Name&gt;

You are a **&lt;role description&gt;**...

## Your Purpose
&lt;What the agent does, when, and for whom&gt;
```

```
## Tone &amp;amp; Style
&amp;lt;Personality guidelines: professional, concise, friendly, etc.&amp;gt;

## Workflow Orchestration

You have access to these skills:
- `&amp;lt;skill-1&amp;gt;`: &amp;lt;what it does&amp;gt;
- `&amp;lt;skill-2&amp;gt;`: &amp;lt;what it does&amp;gt;

### When Triggered by Cron
1. Set RUN_ID: export RUN_ID=$(uuidgen)
2. Execute steps in order:
   - Step 1: Read skills/&amp;lt;skill-1&amp;gt;/SKILL.md, execute via exec()
   - Step 2: Read skills/&amp;lt;skill-2&amp;gt;/SKILL.md, execute via exec()
   - Final Step: Use message() tool for delivery

### Error Handling
&amp;lt;What to do when a step fails&amp;gt;

### Manual Triggers
&amp;lt;What user queries the agent should respond to&amp;gt;

## Key Behaviors
&amp;lt;Domain-specific rules the agent must follow&amp;gt;

## Data Flow
&amp;lt;ASCII diagram showing data pipeline&amp;gt;

## Privacy
&amp;lt;Data handling rules, what NOT to log&amp;gt;
```

Critical rules for SOUL.md

* Workflow steps must reference skill folder names exactly as they appear in skills/.
* Each step should say 'Read the SKILL.md and execute via exec()' - this is the OpenClaw-native pattern.
* The final delivery step uses message() tool, not a standalone API call.
* Error handling must prevent partial outputs - if step N fails, don't proceed to step N+1.

2b. workspace/IDENTITY.md - Agent Identity Card

Role: Lightweight metadata file that gives the agent a recognizable identity within the OpenClaw UI and logs.

How OpenClaw uses it: Loaded into context for branding. Used in UI display, log attribution, and multi-agent environments to distinguish agents.

What to include

```
# &amp;lt;Agent Name&amp;gt; Identity

- **Name:** &amp;lt;Display Name&amp;gt;
- **Agent ID:** &amp;lt;matches agentId in openclaw.json&amp;gt;
- **Emoji:** &amp;lt;single emoji&amp;gt;
- **Vibe:** &amp;lt;3-4 adjective personality summary&amp;gt;
- **Role:** &amp;lt;one-line role&amp;gt;

## What I Do
&amp;lt;2-4 bullet summary of the workflow&amp;gt;

## My Promise
&amp;lt;What the user can expect from this agent&amp;gt;
```

---

3. skills/ - Agent Capabilities

Each skill lives in its own subdirectory under `skills/` and contains exactly one file: `SKILL.md`.

How OpenClaw uses skills

1. SOUL.md tells the agent: 'Read skills//SKILL.md'
2. The agent reads the SKILL.md file
3. The agent extracts the bash/python commands from the code blocks
4. The agent executes them via the exec() tool
5. Results flow to the next step

SKILL.md anatomy

```
# &lt;Skill Name&gt;

## Purpose
&lt;What this skill does in one sentence&gt;

## Dependencies
&lt;CLI tools, packages, env vars needed&gt;

## Workflow

### Step 1: &lt;Action&gt;
&lt;Explanation&gt;
```bash
<executable command>
```

Step 2: <Action>
<Explanation>
```python
<executable script>
```

Expected Output
<What this skill produces and where (table, file, stdout)>

Error Handling
<What to check if it fails>
```

Rules for skills

Rule	Why
Inline exec only - all logic lives in bash/python code blocks inside SKILL.md	OpenClaw agents execute via exec(). No external run.py, main.py, or setup.sh files.
One SKILL.md per folder	OpenClaw discovers skills by scanning for <dir>/SKILL.md.
Folder name = skill ID	The folder name in skills/ must match the string in openclaw.json agents[].skills.
No hardcoded credentials	Use $ENV_VAR references. OpenClaw injects env vars at runtime.
Idempotent operations	Use upsert (not insert). Skills may re-run on retry.
Include RUN_ID	Every data write must include run_id for traceability.
Cleanup temp files	Any /tmp/ files created must be removed at end of skill.

Always-included skill: data-ingestion-openclaw

Every agent that reads/writes structured data must include this skill. It provides the complete API reference for the OpenClaw Data Ingestion Service, including:

* Health & discovery endpoints

* Schema provisioning (one-time setup)
* Data write contract (metrics, artifacts, narratives, generic, batch)
* Query endpoints
* Available tables (entity_* and result_*)
* Python helper patterns

You do not write this file from scratch. Copy the standard data-ingestion-openclaw/SKILL.md from the template repository. It is shared across all agents.

---

4. cron/ - Scheduled Triggers

Contains JSON files that define when OpenClaw should automatically trigger the agent.

Schema

```
{
  "name": "&lt;Human-readable job name&gt;",
  "enabled": true,
  "schedule": {
    "kind": "cron",
    "expr": "&lt;5-field cron expression in UTC&gt;",
    "tz": "UTC"
  },
  "sessionTarget": "isolated",
  "payload": {
    "kind": "agentTurn",
    "message": "&lt;The prompt sent to the agent&gt;",
    "model": "openrouter/anthropic/claude-sonnet-4.5",
    "timeoutSeconds": 600
  }
}
```

Key fields

Field	Notes
schedule.expr	Standard 5-field cron. Must be in UTC. Convert your desired local time. Example: 8:00 AM IST = "30 2 * * *"
schedule.tz	Always "UTC".
sessionTarget	"isolated" = each run gets a fresh session. Use this for stateless scheduled jobs.
payload.message	This exact string is what the agent receives. SOUL.md should have a matching "When Triggered by Cron" section.
payload.model	The LLM model the agent uses. Default: openrouter/anthropic/claude-sonnet-4.5.
payload.timeoutSeconds	Max execution time. 600 (10 min) is typical. Increase for heavy workloads.

Rules

* File name should be descriptive: daily-digest.json, weekly-report.json, etc.
* One JSON file per cron job. An agent can have multiple cron jobs.
* Set "enabled": false during development/testing.

---

5. workflows/main.yaml - Lobster Workflow DAG

Role: Declarative workflow definition using the Lobster format. Defines the execution graph - which skills run, in what order, with what dependencies and timeouts.

How OpenClaw uses it: The platform reads this file to understand the workflow DAG. While SOUL.md drives the actual LLM execution, main.yaml provides the platform with metadata for monitoring, timeout enforcement, and dependency validation.

Schema

```
name: &lt;agent-id&gt;
version: 1.0.0
description: "&lt;what the workflow does&gt;"

trigger:
  cron: "&lt;same cron expression as cron/*.json&gt;"

steps:
  - id: &lt;step-id&gt;
    name: "&lt;Human-readable step name&gt;"
    skill: &lt;skill-folder-name&gt;
    timeout: &lt;duration&gt;  # e.g., 120s, 5m

  - id: &lt;next-step-id&gt;
    name: "&lt;Step name&gt;"
    skill: &lt;skill-folder-name&gt;
    depends_on: [&lt;previous-step-id&gt;]
    timeout: &lt;duration&gt;

  - id: finalize
    name: "Mark Run Complete"
    command: echo "Run complete - RUN_ID=${RUN_ID}"
    depends_on: [&lt;last-step-id&gt;]
```

Rules

* id values must be unique within the workflow.

* skill must match a folder name in skills/.

* depends_on creates the DAG edges. Steps without dependencies run first (or in parallel if multiple have no deps).

* Always end with a finalize step.

* trigger.cron should match cron/*.json - they are the same schedule from two perspectives.

---

6. .env.example - Environment Variable Template

Role: Documents every environment variable the agent needs. Developers copy this to .env and fill in real values.

How OpenClaw uses it: OpenClaw loads .env from its root directory and injects variables into the agent's execution environment.

Format

```
# === Required ===
LINEAR_API_KEY=lin_api_xxxxxxxxxxxxxxxxxxxx
DATA_INGESTION_BASE_URL=https://ingestion-service-s45p.onrender.com
DATA_INGESTION_ORG_ID=your-org-id
DATA_INGESTION_AGENT_ID=&lt;matches agentId in openclaw.json&gt;

# === Optional ===
TELEGRAM_CHAT_ID=123456789
DRY_RUN=false
```

```
RUN_ID=  # auto-generated at runtime
```

Rules

* Never commit real values. .env must be in .gitignore.
* Mark variables as Required or Optional with comments.
* DATA_INGESTION_AGENT_ID must match agentId in openclaw.json.

---

7. Developer Helper Scripts (Optional)

These are convenience scripts for developers during setup and testing. They are **not used by OpenClaw at runtime**.

Script	Purpose
check-environment.sh	Validates all prerequisites: binaries installed, env vars set, services reachable, API keys valid. Run before first deploym
install-dependencies.sh	Installs system dependencies (curl, jq, python3, any CLI tools). Detects OS and uses appropriate package manager.
test-workflow.sh	Simulates the full workflow end-to-end outside of OpenClaw. Useful for debugging individual steps.

Rules

* These scripts must be self-contained and safe to re-run.
* They should never modify OpenClaw state or configuration.
* They are optional but strongly recommended for developer experience.

Deployment Checklist

When you've generated all files for a new agent, verify:

- [ ] openclaw.json exists with valid agentId and skill list
- [ ] workspace/SOUL.md exists with workflow orchestration section
- [ ] workspace/IDENTITY.md exists with agent metadata
- [ ] Every skill listed in openclaw.json has a matching skills//SKILL.md
- [ ] data-ingestion-openclaw/SKILL.md is included (if agent uses data ingestion)
- [ ] All SKILL.md files use inline exec (bash/python code blocks, no external scripts)
- [ ] cron/*.json exists (if agent has scheduled triggers)
- [ ] workflows/main.yaml exists with correct step dependencies
- [ ] cron.payload.message matches what SOUL.md expects
- [ ] .env.example documents all required environment variables
- [ ] All data writes use upsert with run_id
- [ ] No hardcoded credentials anywhere

How Files Map to the OpenClaw Ecosystem

```
Developer creates files          OpenClaw reads &amp;amp; uses them
========================         ============================

openclaw.json          ------&amp;gt;   Agent registration &amp;amp; skill discovery
                                 Platform knows this agent exists

workspace/SOUL.md      ------&amp;gt;   Loaded into LLM context on every trigger
```

```
                                Agent follows these instructions to orchestrate

workspace/IDENTITY.md  ------&gt;   Loaded into LLM context for branding
                                UI display, log attribution

skills/*/SKILL.md      ------&gt;   Read by agent (via SOUL.md instructions)
                                Executed inline via exec() tool

cron/*.json            ------&gt;   Platform scheduler reads these
                                Fires agent at specified intervals

workflows/main.yaml    ------&gt;   Platform reads for DAG metadata
                                Monitoring, timeouts, dependency tracking

.env                   ------&gt;   Platform injects into agent environment
                                Skills access via $ENV_VAR
```

Creating a New Agent - Step by Step

1. Choose an agentId (kebab-case, globally unique)
2. Define the workflow - what steps does the agent perform, in what order?
3. Create openclaw.json - register the agent and list its skills
4. Write workspace/SOUL.md - define persona + workflow orchestration
5. Write workspace/IDENTITY.md - give it a name and personality
6. Create skill folders - one skills//SKILL.md per capability, with inline bash/python
7. Copy data-ingestion-openclaw/SKILL.md from the template (if using data ingestion)
8. Create cron/.json (if scheduled)
9. Create workflows/main.yaml - define the step DAG
10. Create .env.example - document all environment variables
11. Add helper scripts - check-environment.sh, install-dependencies.sh, test-workflow.sh
12. Run the deployment checklist above
13. Copy files to OpenClaw instance and restart gateway

FAQ

Q: Can a skill call another skill?

No. Skills are independent units. Orchestration (ordering, data passing) is handled by SOUL.md.

Q: Can an agent have multiple cron jobs?

Yes. Create multiple JSON files in cron/. Each can trigger the same or different workflows.

Q: Where does the agent store intermediate data?

In the Data Ingestion Service. Entity tables (entity_*) hold raw data. Result tables (result_*) hold processed outputs. Temp files in /tmp/ are used for inter-step data within a single run and must be cleaned up.

Q: What LLM model does the agent use?

Defined in cron/*.json payload.model. Default is openrouter/anthropic/claude-sonnet-4.5.

Q: How do I test without sending real messages?

Set DRY_RUN=true in .env. Skills that support it will log output instead of sending.

Q: Can I use languages other than bash/python in SKILL.md?

Bash and Python are the standard. The exec() tool supports both. If your skill needs Node.js, use bash to invoke node -e "...".